# 🚀 Hướng Dẫn Sử Dụng Continue Extension

## Tối ưu cho Backend Development với Golang & MongoDB

## 📋 Mục lục



## 🎯 Giới thiệu

Continue là AI coding assistant tích hợp VSCode, cho phép:

- **Chat với AI** về code
- **Autocomplete thông minh** (Tab completion)
- **Inline editing** (Ctrl/Cmd + I)
- **Code review tự động**
- **Generate code** từ mô tả

## Mô hình đã được config:

| Mô hình | Mục đích | Shortcut |
|---|---|---|
| **Qwen2.5-Coder-7B** | Chat, Review, Autocomplete | Mặc định |
| **DeepSeek-Coder-V2-16B** | Deep analysis, Architecture | Chọn trong dropdown |
| **nomic-embed-text** | Codebase search | Auto |

# ⚡ Các tính năng chính

## 1. Tab Autocomplete (Quan trọng nhất!)

**Cách sử dụng:**

```
// Gõ dòng comment hoặc function signature, nhấn Tab
// Example: Gõ comment sau và nhấn Tab

// Function to create a new user in MongoDB
func CreateUser(ctx context.Context, db *mongo.Database, user *User) error {
    // [Nhấn Tab ở đây] → AI sẽ generate toàn bộ function body
}
```

**Khi nào dùng:**

- ✅ Viết function mới từ comment/signature
- ✅ Implement interface methods
- ✅ Generate boilerplate code
- ✅ Complete struct definitions

**Settings tối ưu:**

- Temperature: 0.1 (để code nhất quán)
- Stop tokens: Ngừng tại function/type declarations
- Max tokens: 256 (đủ cho 1 function nhỏ)

# 2. Inline Edit (Ctrl/Cmd + I)

**Cách sử dụng:**

1. Highlight code cần sửa
2. Nhấn `Ctrl+I` (Windows/Linux) hoặc `Cmd+I` (Mac)
3. Gõ instruction, ví dụ:
    - "Add error handling"
    - "Optimize this MongoDB query"
    - "Add context timeout"
    - "Convert to table-driven test"

**Use cases:**

```
// Before
func GetUser(id string) (*User, error) {
    filter := bson.M{"_id": id}
    var user User
    err := collection.FindOne(context.Background(), filter).Decode(&user)
    return &user, err
}

// Highlight code → Ctrl+I → "Add context timeout and proper error handling"
// After (AI sẽ generate):
func GetUser(ctx context.Context, id string) (*User, error) {
    ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
    defer cancel()

    filter := bson.M{"_id": id}
    var user User

    err := collection.FindOne(ctx, filter).Decode(&user)
    if err != nil {
        if err == mongo.ErrNoDocuments {
            return nil, fmt.Errorf("user not found: %s", id)
        }
        return nil, fmt.Errorf("failed to get user: %w", err)
    }

    return &user, nil
}
```

# 3. Chat Sidebar (Ctrl/Cmd + L)

**Mở chat:**

- Shortcut: `Ctrl+L` (Windows/Linux) hoặc `Cmd+L` (Mac)
- Hoặc click icon Continue ở sidebar

**Chat patterns hiệu quả:**

```
1. Code Review:
   "Review function này để tìm potential bugs"
   → Attach code bằng @Code

2. Giải thích:
   "Explain how this aggregation pipeline works"

3. Debug:
   "Why is this goroutine leaking? @Terminal"
   → Kèm terminal output

4. Generate:
   "Generate a CRUD service for Product entity with MongoDB"
```

**Context Providers** (Quan trọng!):

| Provider | Cách dùng | Mục đích |
|---|---|---|
| `@Code` | `@Code handler.go` | Attach file cụ thể |
| `@Codebase` | `@Codebase auth service` | Search trong toàn project |
| `@Terminal` | `@Terminal` | Kèm terminal output |
| `@Problems` | `@Problems` | Kèm VSCode errors |
| `@Folder` | `@Folder models/` | Attach toàn bộ folder |

**Ví dụ chat thực tế:**

```
User: "@Code user_handler.go @Codebase user service
Review code này và suggest improvements"

AI: Phân tích code và suggest:
1. Missing context timeout
2. No validation for input
3. MongoDB query không có index
...với code examples
```

# 🔧 Custom Commands chi tiết

## Command 1: `/review-go`

**Mục đích:** Code review toàn diện cho Go code

**Cách dùng:**

1. Highlight code cần review
2. Gõ: `/review-go` trong chat
3. Enter

**AI sẽ kiểm tra:**

- ✅ Error handling patterns
- ✅ Context usage và cancellation
- ✅ Race conditions
- ✅ MongoDB query optimization
- ✅ Memory leaks
- ✅ Idiomatic Go

**Example:**

```
// Highlight đoạn code này và chạy /review-go
func (s *UserService) UpdateUser(id string, updates map[string]interface{}) error {
    filter := bson.M{"_id": id}
    update := bson.M{"$set": updates}
    _, err := s.collection.UpdateOne(context.Background(), filter, update)
    return err
}

// AI sẽ chỉ ra:
// ❌ Không có context timeout
// ❌ Không validate updates input
// ❌ Không check if document exists
// ❌ Error message không rõ ràng
// ✅ Suggest code cải thiện
```

## Command 2: `/optimize-mongo`

**Mục đích:** Tối ưu MongoDB queries

**Cách dùng:**

```
// Highlight query code và run /optimize-mongo
pipeline := mongo.Pipeline{
    bson.D{{"$match", bson.D{{"status", "active"}}}},
    bson.D{{"$lookup", bson.D{
        {"from", "orders"},
        {"localField", "_id"},
        {"foreignField", "user_id"},
        {"as", "orders"},
    }}},
    bson.D{{"$unwind", "$orders"}},
}

// AI sẽ analyze và suggest:
// 1. Index recommendations: {status: 1}, {user_id: 1}
// 2. Thêm $project để giảm data transfer
// 3. Use $match sau $lookup nếu filter orders
// 4. Consider pagination với $skip/$limit
```

**Best practices AI sẽ check:**

- Index coverage
- Pipeline stage order
- Memory usage ($sort$,group)
- Network payload size
- Aggregation optimization

## Command 3: `/add-tests`

**Mục đích:** Generate unit tests tự động

**Cách dùng:**

```
// Highlight function cần test
func (s *UserService) CreateUser(ctx context.Context, user *User) error {
    if user.Email == "" {
        return errors.New("email is required")
    }

    user.CreatedAt = time.Now()
    _, err := s.collection.InsertOne(ctx, user)
    return err
}

// Run /add-tests
```

**AI sẽ generate:**

```
func TestUserService_CreateUser(t *testing.T) {
    tests := []struct {
        name    string
        user    *User
        wantErr bool
        errMsg  string
    }{
        {
            name: "valid user",
            user: &User{
                Email: "test@example.com",
                Name:  "Test User",
            },
            wantErr: false,
        },
        {
            name: "missing email",
            user: &User{
                Name: "Test User",
            },
            wantErr: true,
            errMsg:  "email is required",
        },
        // ... more test cases
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            // Setup mock MongoDB
            // Run test
            // Assert results
        })
    }
}
```

## Command 4: `/gen-handler`

**Mục đích:** Generate HTTP handler hoàn chỉnh

**Cách dùng:**

```
/gen-handler

API endpoint: POST /api/users
Input: {name, email, password}
Action: Create user in MongoDB
Validation: email format, password min 8 chars
Response: User object without password
```

**AI sẽ generate:**

```go
type CreateUserRequest struct {
    Name     string `json:"name" validate:"required"`
    Email    string `json:"email" validate:"required,email"`
    Password string `json:"password" validate:"required,min=8"`
}

type CreateUserResponse struct {
    ID        string    `json:"id"`
    Name      string    `json:"name"`
    Email     string    `json:"email"`
    CreatedAt time.Time `json:"created_at"`
}

func (h *UserHandler) CreateUser(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 5*time.Second)
    defer cancel()

    var req CreateUserRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid request body", http.StatusBadRequest)
        return
    }

    if err := h.validator.Struct(req); err != nil {
        http.Error(w, fmt.Sprintf("Validation error: %v", err), http.StatusBadRequest)
        return
    }

    // Hash password
    hashedPassword, err := bcrypt.GenerateFromPassword([]byte(req.Password), bcrypt.Def
    if err != nil {
        h.logger.Error("Failed to hash password", "error", err)
        http.Error(w, "Internal server error", http.StatusInternalServerError)
        return
    }

    user := &User{
        ID:        primitive.NewObjectID(),
        Name:      req.Name,
        Email:     req.Email,
        Password:  string(hashedPassword),
        CreatedAt: time.Now(),
    }
```

```
    if _, err := h.collection.InsertOne(ctx, user); err != nil {
        h.logger.Error("Failed to insert user", "error", err)
        http.Error(w, "Failed to create user", http.StatusInternalServerError)
        return
    }

    resp := CreateUserResponse{
        ID:        user.ID.Hex(),
        Name:      user.Name,
        Email:     user.Email,
        CreatedAt: user.CreatedAt,
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(resp)
}
```

# 💼 Workflow thực tế

## Workflow 1: Implement tính năng mới

1. Design phase:
   Chat: "I need to implement user authentication with JWT.
          Suggest architecture for @Codebase"

2. Generate models:
   /gen-model
   "User model với fields: email, password_hash, role, tokens"

3. Generate handler:
   /gen-handler
   "POST /login endpoint với JWT generation"

4. Add tests:
   Highlight generated code → /add-tests

5. Review:
   Highlight all new code → /review-go

6. Optimize:
   Highlight MongoDB queries → /optimize-mongo

## Workflow 2: Debug lỗi

1. Chạy code → Có lỗi trong terminal

2. Mở Continue chat:
   "@Terminal @Code user_service.go
   Why am I getting this error?"

3. AI analyze error và suggest fix

4. Apply fix với Inline Edit (Ctrl+I)

5. Verify với /review-go

## Workflow 3: Refactor code

1. Highlight code cần refactor

2. Ctrl+I → "Refactor theo clean architecture:
   - Separate business logic
   - Add dependency injection
   - Improve testability"

3. AI sẽ generate refactored code

4. Review changes

5. /add-tests để ensure không break functionality

# 🎯 Best Practices

## 1. Context là chìa khóa

❌ Bad:
"Fix this code"

✅ Good:
"@Code handler.go @Codebase user service
This handler is returning 500 error when email exists.
@Terminal shows 'duplicate key error'.
How to handle this gracefully?"

## 2. Incremental changes

```
Thay vì:
"Rewrite toàn bộ service này"

Nên:
1. "Add context timeout" → Apply
2. "Add input validation" → Apply
3. "Add error logging" → Apply
4. "Optimize MongoDB query" → Apply
```

## 3. Sử dụng đúng model cho đúng task

| Task | Model | Lý do |
|---|---|---|
| Quick autocomplete | Qwen2.5-Coder-7B | Nhanh, chính xác |
| Code review | Qwen2.5-Coder-7B | Balanced |
| Architecture design | DeepSeek-V2-16B | Deep analysis |
| Generate boilerplate | Qwen2.5-Coder-7B | Consistent output |

## 4. Verify AI output

```
Luôn luôn:
✅ Review generated code
✅ Run tests
✅ Check for security issues
✅ Validate business logic
✅ Test edge cases
```

## 5. Custom commands cho common tasks

Tạo thêm commands cho patterns hay dùng:

```
{
  "name": "add-logging",
  "prompt": "Add structured logging using slog to this code:\n{{{ input }}}",
  "description": "Add logging statements"
}
```

# 🔧 Troubleshooting

## Vấn đề 1: Autocomplete không hoạt động

**Nguyên nhân:**

- Ollama service không chạy
- Model chưa được pull
- Conflict với extensions khác (Copilot, Tabnine)

**Giải pháp:**

```
# Check Ollama
ps aux | grep ollama

# Restart Ollama
pkill ollama
ollama serve

# Verify model
ollama list | grep qwen2.5-coder

# Disable conflicting extensions trong VSCode
```

## Vấn đề 2: Response chậm

**Nguyên nhân:**

- GPU memory đầy
- Model quá lớn cho VRAM
- Context quá dài

**Giải pháp:**

```
// Giảm context length trong config.json
{
  "contextLength": 8192,  // Từ 32768 xuống
  "completionOptions": {
    "numPredict": 1024    // Từ 2048 xuống
  }
}
```

## Vấn đề 3: AI generate code không đúng

**Nguyên nhân:**

- Context không đủ rõ ràng
- Temperature quá cao
- Model không phù hợp với task

**Giải pháp:**

1. Cung cấp context đầy đủ hơn
2. Giảm temperature xuống 0.1-0.2
3. Sử dụng system message rõ ràng hơn

## Vấn đề 4: Codebase search không chính xác

**Giải pháp:**

```
# Re-index codebase
# Trong VSCode Command Palette (Ctrl+Shift+P):
> Continue: Index Codebase

# Hoặc delete embeddings cache
rm -rf ~/.continue/index
```

# 🚀 Advanced Configuration

## 1. Tối ưu cho Performance

```
{
  "tabAutocompleteModel": {
    "completionOptions": {
      "temperature": 0.05,     // Càng thấp càng consistent
      "topK": 20,              // Giảm để faster
      "numPredict": 128,       // Chỉ generate ngắn
      "repeatPenalty": 1.1     // Tránh lặp lại
    }
  }
}
```

## 2. Multi-model Strategy

```
{
  "models": [
    {
      "title": "Fast (7B)",
      "model": "qwen2.5-coder:7b"
    },
    {
      "title": "Smart (16B)",
      "model": "deepseek-coder-v2:16b-lite-instruct-q4_K_M"
    }
  ]
}
```

**Khi nào dùng model nào:**

- **Fast (7B)**: Autocomplete, quick chat, simple refactoring
- **Smart (16B)**: Architecture design, complex debugging, optimization

## 3. Custom System Messages

Tùy chỉnh theo project:

```
{
  "systemMessage": "You are an expert in:
    - Go microservices with gRPC
    - MongoDB with replica sets
    - Redis caching patterns
    - Docker containerization

    Follow these conventions:
    - Use context.Context in all functions
    - Implement graceful shutdown
    - Add OpenTelemetry tracing
    - Use structured logging with slog"
}
```

## 4. Keyboard Shortcuts tối ưu

**File: `.vscode/keybindings.json`**

```
[
  {
    "key": "ctrl+shift+l",
    "command": "continue.continueGUIView.focus"
  },
  {
    "key": "ctrl+shift+i",
    "command": "continue.acceptDiff"
  },
  {
    "key": "ctrl+shift+k",
    "command": "continue.rejectDiff"
  }
]
```

## 5. Project-specific Config

**File: `.continue/config.json` (trong project root)**

```
{
  "contextProviders": [
    {
      "name": "folder",
      "params": {
        "folders": ["internal/", "pkg/", "cmd/"]
      }
    }
  ],
  "docs": [
    {
      "title": "Internal Wiki",
      "startUrl": "http://wiki.company.com/backend"
    }
  ]
}
```

# 📊 Metrics & Monitoring

## Track productivity gains:

1. **Time saved per day**: Estimate trước/sau khi dùng Continue
2. **Code quality**: Số bugs giảm sau code review
3. **Test coverage**: Increase từ generated tests
4. **Learning curve**: Time để học API/patterns mới

## Example metrics:

```
Before Continue:
- Write CRUD handler: ~30 mins
- Write unit tests: ~20 mins
- Code review findings: ~5 issues/PR
- Debug time: ~1 hour/issue

After Continue:
- Write CRUD handler: ~10 mins (Generate + Review)
- Write unit tests: ~5 mins (Auto-generate)
- Code review findings: ~2 issues/PR (/review-go trước)
- Debug time: ~20 mins (AI-assisted debugging)

Total time saved: ~50-60% cho routine tasks
```

# 🎓 Learning Path

## Week 1: Basics

- [ ] Setup và config
- [ ] Practice tab autocomplete
- [ ] Try basic chat queries
- [ ] Use 2-3 custom commands

## Week 2: Intermediate

- [ ] Master inline editing
- [ ] Use context providers
- [ ] Create custom commands
- [ ] Integrate vào daily workflow

## Week 3: Advanced

- [ ] Multi-model strategy
- [ ] Optimize config cho project
- [ ] Create project-specific commands

- [ ] Share best practices với team

# 🆘 Support & Resources

## Documentation:

- Continue Docs: https://continue.dev/docs
- Ollama Models: https://ollama.com/library
- Go Best Practices: https://go.dev/doc/effective_go

## Community:

- Continue Discord: https://discord.gg/continue
- GitHub Issues: https://github.com/continuedev/continue

## Company Support:

- Internal Slack: #ai-coding-tools
- Wiki: [Link to company wiki]

# ✅ Checklist trước khi bắt đầu

- [ ] Ollama installed và running
- [ ] Models đã được import
- [ ] Continue extension installed
- [ ] Config file applied
- [ ] Test với 1 simple command
- [ ] Đọc qua docs này
- [ ] Setup keyboard shortcuts
- [ ] Join community channels

🎉 **Chúc bạn code vui vẻ và hiệu quả với Continue!**

*Last updated: 2025-10-20*